

Projeto Final

Valor: 30 (+2) pontos

Data de entrega: 27/11/2025

1. Introdução

O grupo deve escolher um problema de seu interesse e realizar todo o processo de desenvolvimento (análise, projeto e implementação) de uma solução. Espera-se ao final um sistema de porte médio (>2000 linhas de código) que utiliza os conceitos e técnicas vistos durante o curso (modelagem, POO, testes unitários, etc). O programa deve ser feito baseado na linguagem C++11.

Uma lista não exaustiva ou exclusiva de sugestões de temas é apresentada abaixo. O tema é aberto à negociação e os alunos também podem sugerir outros temas de seu interesse.

- Twitter (tweet, retweet, follow, etc)
- Jogo de cartas (distribuir cartas, fazer jogadas, alternar turnos, etc)
- Gerenciador de tarefas/compromissos (adicionar, remover, listar, histórico, etc)
- Ecommerce (cadastrar produtos, buscar, comprar, etc)
- Mini rede social (timeline, adicionar/listar amigos, post, etc)
- UFMG: Carona, Eventos, Bandeirão, Gamefication, Oportunidades, ...
- Outros (Magic, RPG, Reserva de Passagens/Hotel, Netflix, Spotify, Trello/Notion, ...)

ATENÇÃO: Temas similares poderão ser escolhidos por no máximo 3 grupos!

O desenvolvimento e a entrega deverão ser feitos utilizando o sistema de controle de versão GitHub¹. Sugere-se que commits/pushs sejam feitos de maneira frequente, sempre que houver alterações.

O calendário de atividades do trabalho é mostrado abaixo:

Atividade	Data
Formação do Grupo (planilha Moodle)	Até 26/08/25
Definição do tema (planilha Moodle)	Até 28/08/25
Apresentação / Entrevista	25/11/25 e 27/11/25
Entrega final (Github)	27/11/25

Lembre-se, o objetivo não é apenas escrever um programa funcional, mas **desenvolver um sistema confiável, reutilizável e de fácil manutenção e extensão!** Logo, tente **aplicar todos os conceitos de POO, modularidade e corretude** vistos em sala de aula. Também serão avaliados critérios como criatividade na solução, assim como a possível implementação de funcionalidades extras.

¹ <https://github.com/>

Durante o desenvolvimento, o grupo deverá utilizar o framework doctest² para implementar os testes de unidade. Deve haver pelo menos uma classe de testes para cada uma das principais classes do sistema (por exemplo, as classes de entidades). Além disso, deve-se apresentar uma cobertura total do código de pelo menos **60%** (a ser verificado utilizando a ferramenta gcovr³).

A interface de interação com a aplicação poderá ser feita via terminal de comando. Possíveis arquivos necessários durante a etapa de inicialização do sistema deverão ser fornecidos pelo grupo.

2. Requisitos e Modelagem

As User Stories⁴ são uma forma simples de apresentar os requisitos funcionais desejados para um determinado sistema. São artefatos de desenvolvimento utilizados principalmente em processos baseados em metodologias ágeis. As descrições são intencionalmente genéricas, dando liberdade ao grupo para decidir detalhes da implementação.

Nesta etapa, o grupo deverá identificar possíveis funcionalidades interessantes de serem incorporadas ao sistema e propor pelo menos **SEIS** User Stories. Para cada uma, deverá ser feita uma descrição sucinta, similar ao exemplo abaixo.

- **Sistema de Gestão Acadêmica**
- **Descrição:**
 - Como professor quero gerar um relatório para verificar o desempenho dos alunos.
- **Critérios de aceitação:**
 - Mostrar a pontuação da avaliação atual de um aluno.
 - Mostrar a pontuação de avaliação passada de um aluno.
 - Mostrar a média e desvio padrão da turma para a avaliação.

Tente apresentar User Stories para **diferentes tipos de usuário do sistema** (ou considerando papéis específicos em certos contextos). Cada User Story deve apresentar entre **3 e 5 critérios de aceitação**.

Em seguida, faça pelo menos **SEIS** cartões CRC para as classes que você julga como sendo as mais importantes no sistema. Cada cartão deve apresentar entre **5 e 10 responsabilidades** (considerando conhecimento e realização) e mencionar pelo menos **2 colaborações**. Você pode usar esse site para lhe auxiliar a fazer o cartão: <https://echeung.me/crcmaker/>

Os Cartões CRC e as User Stories devem ser colocados no próprio **repositório do grupo** no GitHub.

3. Documentação

Todo o desenvolvimento do projeto deverá ser documentado utilizando-se Doxygen⁵ e o arquivo README⁶ do repositório. No código, detalhe as estruturas de dados utilizadas e o funcionamento dos métodos. No README, faça uma breve apresentação do problema, visão geral da solução focando na estrutura e funcionamento do programa, e as principais dificuldades encontradas.

² <https://github.com/onqtam/doctest>

³ <https://gcovr.com/en/stable/>

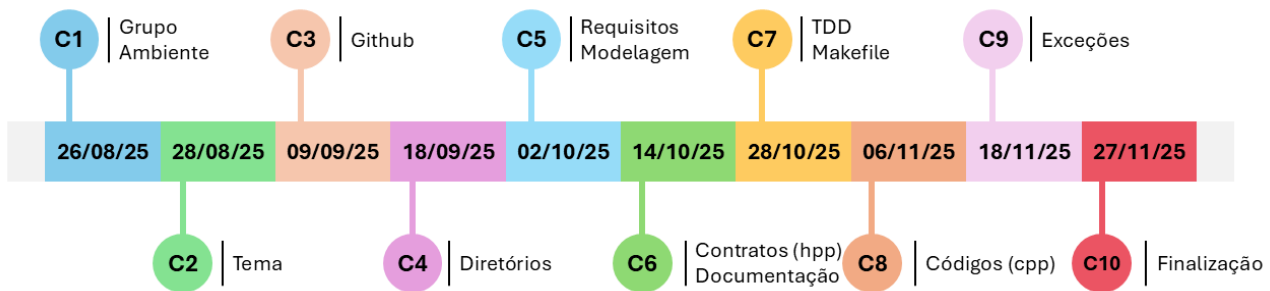
⁴ https://en.wikipedia.org/wiki/User_story

⁵ <http://www.doxygen.org/>

⁶ <https://docs.github.com/pt/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>

4. Checkpoints

O projeto será desenvolvido de forma **incremental**, possibilitando a evolução contínua da solução ao longo do semestre. O acompanhamento do trabalho ocorrerá por meio de **10 checkpoints**, distribuídos ao longo da disciplina, cada um associado a entregáveis específicos que permitem monitorar o progresso e realizar ajustes quando necessário. Os detalhes de cada checkpoint encontram-se descritos a seguir e no cronograma oficial da disciplina. O não cumprimento de um checkpoint acarretará **penalização de 5% na nota final**, cumulativa a cada etapa não atendida.



- **C1 – Formação do Grupo e Configuração do ambiente de desenvolvimento**
 - Preparação do ambiente de trabalho, incluindo instalação do compilador, bibliotecas, IDE e demais ferramentas necessárias para o projeto.
- **C2 – Definição do tema**
 - Escolha e delimitação inicial do escopo do projeto, com breve descrição do problema a ser resolvido e dos objetivos a serem alcançados.
- **C3 – Acesso ao GitHub e Commit de teste**
 - Configuração do repositório no GitHub e realização de um commit de teste para validação do fluxo de versionamento.
- **C4 – Estrutura de diretórios**
 - Organização inicial dos diretórios do projeto (ex.: src/, include/, build/, tests/) seguindo boas práticas de modularização.
- **C5 – Requisitos e modelagem**
 - Levantamento dos requisitos funcionais e não funcionais (User Stories) e elaboração da modelagem inicial (Cartões CRC) que descrevam o sistema.
- **C6 – Definição dos contratos (.hpp) e documentação inicial (README+Doxygen)**
 - Criação dos arquivos de cabeçalho com a especificação das interfaces e início da documentação automatizada usando Doxygen e descrição via README.
- **C7 – Testes de unidade (TDD) e Makefile**
 - Implementação inicial dos testes unitários seguindo a abordagem Test-Driven Development e criação do Makefile para automação de compilação.
- **C8 – Implementação inicial dos comportamentos (.cpp)**
 - Codificação inicial das funcionalidades principais do sistema com base nas interfaces definidas, garantindo conformidade com os requisitos.
- **C9 – Programação defensiva e Tratamento de exceções**
 - Inclusão de mecanismos para aumentar a robustez do código, prevenindo erros e tratando adequadamente condições excepcionais.
- **C10 – Finalização e apresentação**
 - Refinamento final do projeto, revisão de código, entrega da versão definitiva e apresentação dos resultados para a turma.

5. Comentários Gerais

- Comece a fazer o trabalho logo, enquanto o prazo está tão longe quanto jamais poderá estar.
- O trabalho deverá ser feito em grupos com 4 ou 5 alunos.
- Trabalhos copiados serão penalizados conforme anunciado.
- A entrega final será considerada o último commit na branch principal do projeto.
- Deve ser fornecido com o código um arquivo Makefile com as opções ‘make’ e ‘make run’.

6. Critérios de avaliação

6.1. Entrega Parcial (Requisitos e Modelagem)

- User Stories (3 pts).
- Cartões CRC (3 pts).

6.2. Entrega Final (Implementação)

- Documentação (4 pts).
 - Detalhamento do projeto (README).
 - Doxygen.
- Funcionamento correto (4 pts).
 - Compila e executa, não apresenta crash, etc.
- Uso correto das boas práticas e dos conceitos de OO (6 pts).
 - Abstração, Encapsulamento, Herança e Polimorfismo.
 - Modularização e componentes reusáveis.
- Programação defensiva / Tratamento de exceções (3 pts).
- Testes de Unidade / Cobertura (3 pts).
- Apresentação / Entrevista (4 pts).
- Participação individual ([0, 1]).
 - Baseado na quantidade de commits no GitHub.
 - **Grupo é responsável pela distribuição adequada de tarefas entre os membros!**
- Criatividade, extras (+2 pts).

$$\text{NotaFinal} = P \cdot (A \cdot \sum \text{Itens})$$

↑ Boolean de apresentação

↑ Float de participação